Name: Alok Kumar Tarini

Regd. No: 2141020010

System Monitor Tool

Objective: Create a system monitor tool in C++ that displays real-time information about system processes, memory usage, and CPU load, similar to the 'top' command.

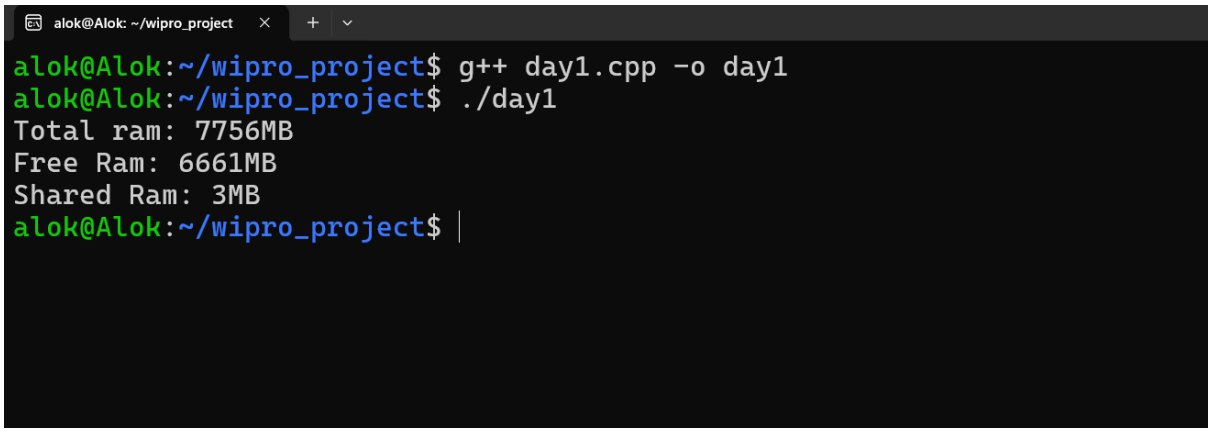
Day-wise Tasks:

Day 1: Design UI layout and gather system data using system calls.

Code:

```
#include<iostream>
using namespace std;
#include<sys/sysinfo.h>

void display_Memory(){
   struct sysinfo info;
   if(sysinfo(&info) == 0){
      cout<<"Total ram: "<<info.totalram / (1024*1024)<<"MB\n";
      cout<<"Free Ram: "<<info.freeram / (1024*1024)<<"MB\n";
      cout<<"Shared Ram: "<<info.sharedram / (1024*1024)<<"MB\n";
   }

}
int main(){
   display_Memory();
   return 0;
}
```

Output:



Day 2: Display process list with CPU and memory usage.

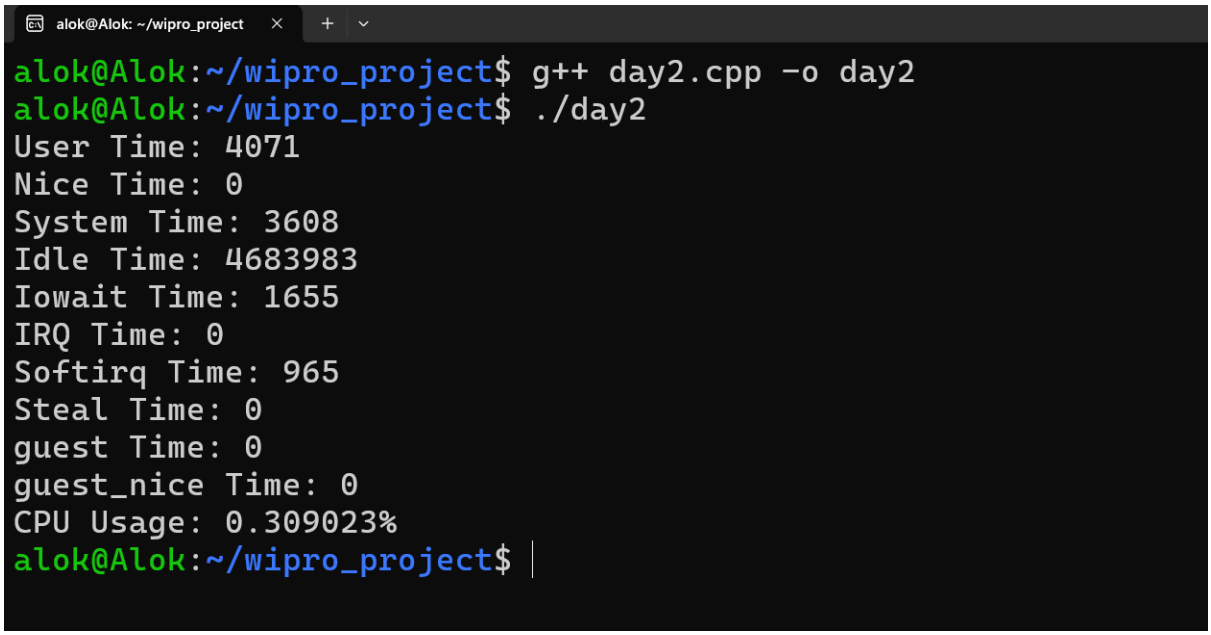
Code:

```
#include<iostream>

#include<fstream>

#include<sstream>

#include<thread>

#include<chrono>


using namespace std;


struct CPU_Data {

   long user, nice, system, idle, iowait, irq, softirq, steal, guest, guest_nice;

};


CPU_Data getCpuData(){

   ifstream file("/proc/stat");

   string line;

   CPU_Data cpu = {};

   if(file.is_open()){

      getline(file, line);
```

```
        istringstream ss(line);
        string CPU_Label;
        ss>>CPU_Label>>cpu.user>>cpu.nice>>cpu.system>>cpu.idle>>cpu.iowait>>cpu.irq
        >>cpu.softirq>>cpu.steal>>cpu.guest>>cpu.guest_nice;
    }
    return cpu;
}

double cpu_use(CPU_Data prev, CPU_Data curr){
    long previdle = prev.idle + prev.iowait;
    long curridle = curr.idle + curr.iowait;

    long prevtotal = prev.user + prev.nice + prev.system + previdle + prev.irq
    + prev.softirq + prev.steal;

    long currtotal = curr.user + curr.nice + curr.system + curridle + curr.irq
    + curr.softirq + curr.steal;

    long totaldiff = currtotal - prevtotal;
    long idlediff = curridle - previdle;

    return (totaldiff - idlediff) * 100.0 / totaldiff;
}

int main(){
    CPU_Data cpu = getCpuData();
    cout<<"User Time: "<<cpu.user<<"\n";
    cout<<"Nice Time: "<<cpu.nice<<"\n";
```

```
    cout<<"System Time: "<<cpu.system<<"\n";
    cout<<"Idle Time: "<<cpu.idle<<"\n";
    cout<<"Iowait Time: "<<cpu.iowait<<"\n";
    cout<<"IRQ Time: "<<cpu.irq<<"\n";
    cout<<"Softirq Time: "<<cpu.softirq<<"\n";
    cout<<"Steal Time: "<<cpu.steal<<"\n";
    cout<<"guest Time: "<<cpu.guest<<"\n";
    cout<<"guest_nice Time: "<<cpu.guest_nice<<"\n";

    CPU_Data prevdata = getCpuData();
    this_thread::sleep_for(chrono::seconds(1));
    CPU_Data currdata = getCpuData();
    double cpuUsage = cpu_use(prevdata, currdata);
    cout<<"CPU Usage: "<<cpuUsage<<"%\n";
    return 0;
}
```

Output:

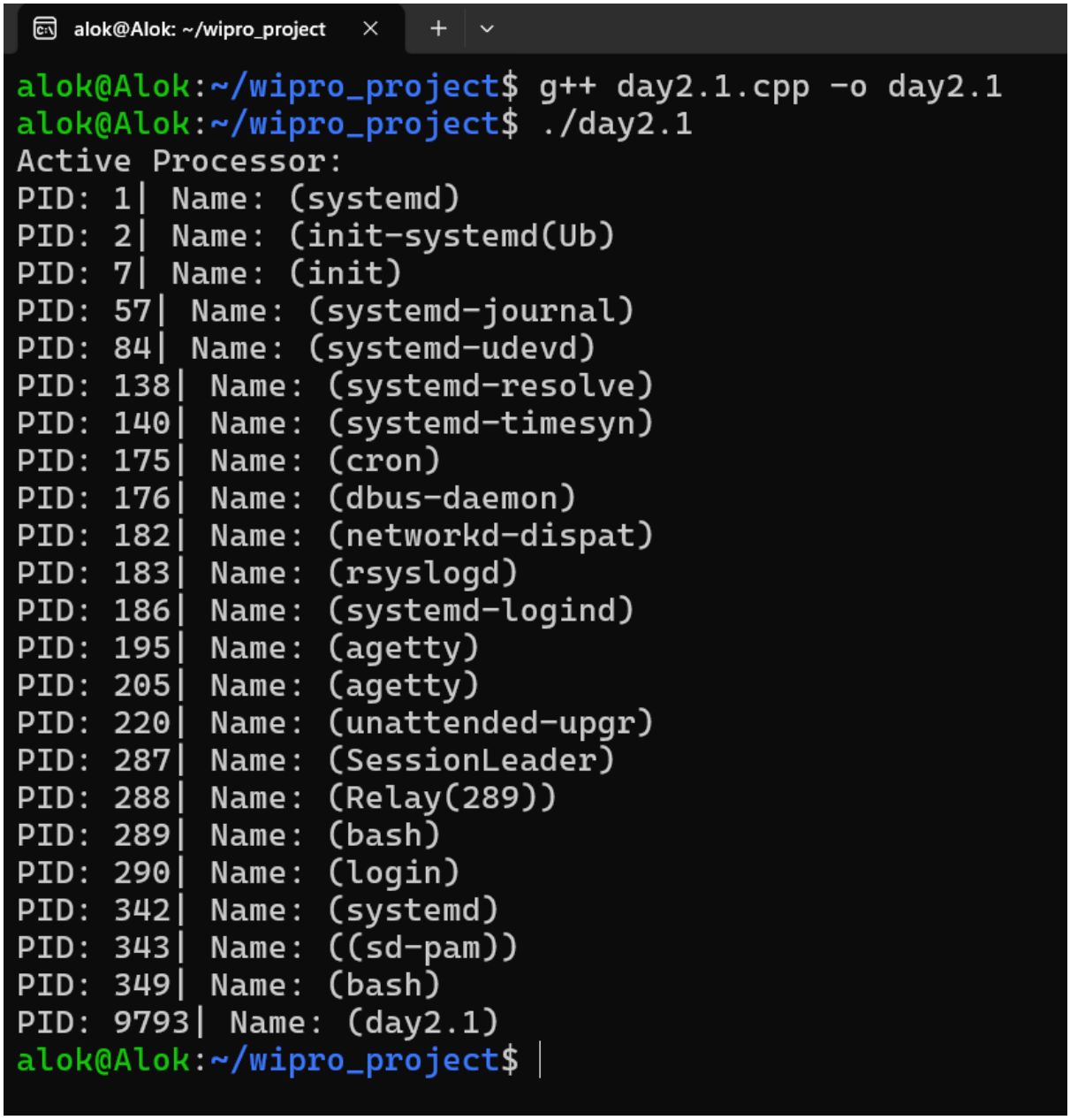


Code:

```
#include<iostream>
#include<filesystem>
#include<fstream>
#include<sstream>
#include<thread>
#include<chrono>
#include<algorithm>
using namespace std;
namespace fs = filesystem;

bool isNumber(const string &s){
   return !s.empty() && all_of(s.begin(), s.end(), ::isdigit);
}

string getProcessName(int pid){
   ifstream file("/proc/" + to_string(pid) + "/stat");
   string line, processName;
   if(file.is_open()){
      getline(file, line);
      istringstream ss(line);
      string token;
      int count = 0;
      while(ss >> token){
         count++;
         if(count == 2){
            processName = token;
            break;
```

```
            }
        }
    }
    return processName;
}

int main(){
    cout<<"Active Processor: \n";
    for(const auto &entry : fs::directory_iterator("/proc")){
        string filename = entry.path().filename().string();
        if(isNumber(filename)){
            int pid = stoi(filename);
            string processName = getProcessName(pid);
            cout<<"PID: "<<pid<<"| Name: "<<processName<<endl;
        }
    }
    return 0;
}
```

Output:


alok@Alok: ~/wipro_project
alok@Alok:~/wipro_project$ g++ day2.1.cpp -o day2.1
alok@Alok:~/wipro_project$ ./day2.1
Active Processor:
PID: 1| Name: (systemd)
PID: 2| Name: (init-systemd(Ub)
PID: 7| Name: (init)
PID: 57| Name: (systemd-journal)
PID: 84| Name: (systemd-udevd)
PID: 138| Name: (systemd-resolve)
PID: 140| Name: (systemd-timesyn)
PID: 175| Name: (cron)
PID: 176| Name: (dbus-daemon)
PID: 182| Name: (networkd-dispat)
PID: 183| Name: (rsyslogd)
PID: 186| Name: (systemd-logind)
PID: 195| Name: (agetty)
PID: 205| Name: (agetty)
PID: 220| Name: (unattended-upgr)
PID: 287| Name: (SessionLeader)
PID: 288| Name: (Relay(289))
PID: 289| Name: (bash)
PID: 290| Name: (login)
PID: 342| Name: (systemd)
PID: 343| Name: ((sd-pam))
PID: 349| Name: (bash)
PID: 9793| Name: (day2.1)
alok@Alok:~/wipro_project$

Day 3: Implement process sorting by CPU and memory usage.

Code:

```
#include <iostream>
#include <filesystem>
#include <fstream>
#include <sstream>
#include <algorithm>
#include <vector>
#include <dirent.h>
#include <unistd.h>
#include <csignal>

using namespace std;

namespace fs=std::filesystem;

struct ProcessInfo{
   int pid;
   string name;
   double cpuUsage;
   long memoryUsage;
};

string readFileValue(const string &path){
   ifstream file(path);
   string value;
   if (file.is_open()){
      getline(file,value);
```

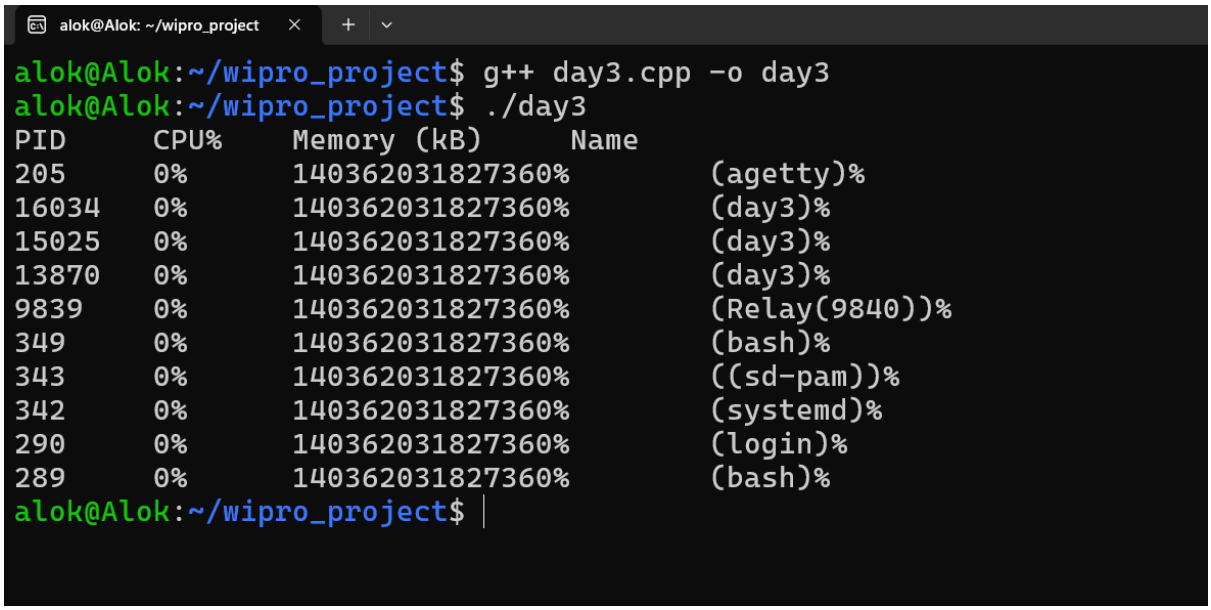
```
    }
    return value;
}

double getSystemUptime(){
    ifstream file("/proc/uptime");
    double uptime;
    if (file.is_open()){
        file >> uptime;
    }
    return uptime;
}

ProcessInfo getProcessInfo(int pid,double systemUptime){
    ProcessInfo pinfo;
    pinfo.pid = pid;
    ifstream file("/proc/" + to_string(pid) + "/stat");
    string line;
    long utime=0, stime=0, starttime=0;
    if(file.is_open()){
        getline(file,line);
        istringstream ss(line);
        string token;
        int count=0;

        while(ss >> token){
            count++;
            if(count==2) pinfo.name=token;
```

```
            else if(count==14) utime=stol(token);
            else if(count==15) stime=stol(token);
            else if(count==22) starttime=stol(token);
        }
    }

    ifstream memFile("/proc/" + to_string(pid) + "/status");
    if (memFile.is_open()){
        string key, value,unit;
        while(memFile >> key >> value >> unit){
            if (key=="VmRSS"){
                pinfo.memoryUsage=stol(value);
                break;
            }
        }
    }
    long total_time = utime + stime;
    double seconds = systemUptime - (starttime / sysconf(_SC_CLK_TCK));
    pinfo.cpuUsage = (total_time / sysconf(_SC_CLK_TCK)) / seconds * 100;
    return pinfo;


}

vector <ProcessInfo> getAllProcesses(){
    vector <ProcessInfo> processes;
    double systemUptime= getSystemUptime();
    for (const auto &entry : fs::directory_iterator("/proc")){
```

```
        if (entry.is_directory()){
            string filename=entry.path().filename().string();
            if (all_of(filename.begin(),filename.end(), ::isdigit)){
                int pid=stoi(filename);
                processes.push_back(getProcessInfo(pid,systemUptime));
            }
        }
    }
    return processes;
}

void sortProcesses(vector<ProcessInfo> &processes, bool sortByCpu){
    if(sortByCpu){
        sort(processes.begin(),processes.end(),[](const ProcessInfo &a, const ProcessInfo &b)
        {
            return a.cpuUsage > b.cpuUsage;
        });
    }
    else{
        sort(processes.begin(),processes.end(),[](const ProcessInfo &a, const ProcessInfo &b)
        {
            return a.memoryUsage > b.memoryUsage;
        });
    }
}



int main(){
```

```
    vector <ProcessInfo> processes = getAllProcesses();
    sortProcesses(processes,true);
    cout<<"PID\tCPU%\tMemory (kB)\tName\n";
    for (size_t i=0;i<min(processes.size(),size_t(10));i++){
        cout<<processes[i].pid<<"\t"<<processes[i].cpuUsage<<"%\t"<<processes[i].memoryUsage<<"%\t"<<processes[i].name<<"%\n";
    }
    return 0;
}
```

Output:



Day 4: Add functionality to kill processes.

Code:

```
#include <iostream>
#include <filesystem>
#include <fstream>
#include <sstream>
#include <algorithm>
#include <vector>
#include <dirent.h>
#include <unistd.h>
#include <csignal>
using namespace std;
namespace fs=std::filesystem;
struct ProcessInfo{
   int pid;
   string name;
   double cpuUsage;
   long memoryUsage;
};

string readFileValue(const string &path){
   ifstream file(path);
   string value;
   if (file.is_open()){
      getline(file,value);
   }
   return value;
}
```

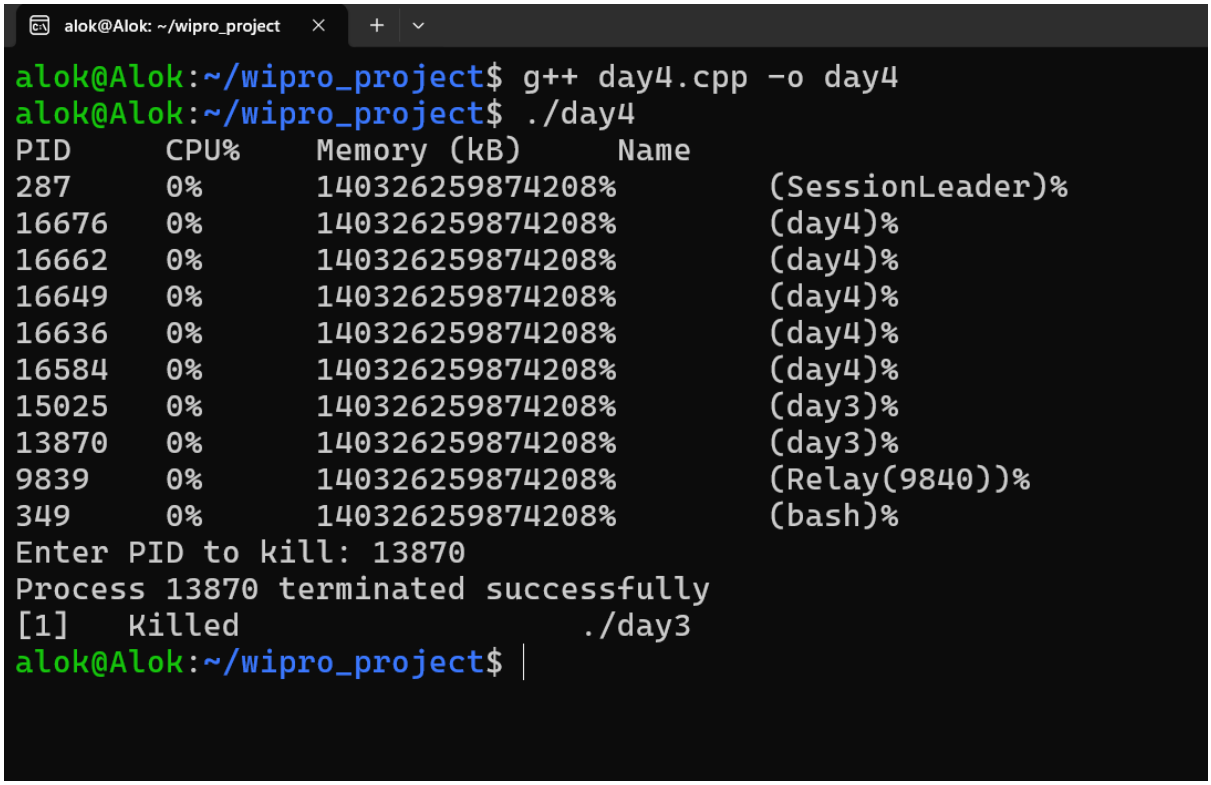
```
double getSystemUptime(){
    ifstream file("/proc/uptime");
    double uptime;
    if (file.is_open()){
        file >> uptime;
    }
    return uptime;
}
ProcessInfo getProcessInfo(int pid,double systemUptime){
    ProcessInfo pinfo;
    pinfo.pid = pid;
    ifstream file("/proc/" + to_string(pid) + "/stat");
    string line;
    long utime=0, stime=0, starttime=0;
    if(file.is_open()){
        getline(file,line);
        istringstream ss(line);
        string token;
        int count=0;

        while(ss >> token){
            count++;
            if(count==2) pinfo.name=token;
            else if(count==14) utime=stol(token);
            else if(count==15) stime=stol(token);
            else if(count==22) starttime=stol(token);
        }
    }
```

```
    ifstream memFile("/proc/" + to_string(pid) + "/status");
    if (memFile.is_open()){
        string key, value,unit;
        while(memFile >> key >> value >> unit){
            if (key=="VmRSS"){
                pinfo.memoryUsage=stol(value);
                break;
            }
        }
    }
    long total_time = utime + stime;
    double seconds = systemUptime - (starttime / sysconf(_SC_CLK_TCK));
    pinfo.cpuUsage = (total_time / sysconf(_SC_CLK_TCK)) / seconds * 100;
    return pinfo;
}
vector <ProcessInfo> getAllProcesses(){
    vector <ProcessInfo> processes;
    double systemUptime= getSystemUptime();
    for (const auto &entry : fs::directory_iterator("/proc")){
        if (entry.is_directory()){
            string filename=entry.path().filename().string();
            if (all_of(filename.begin(),filename.end(), ::isdigit)){
                int pid=stoi(filename);
                processes.push_back(getProcessInfo(pid,systemUptime));
            }
        }
    }
    return processes;
```

```
}
void sortProcesses(vector<ProcessInfo> &processes, bool sortByCpu){
   if(sortByCpu){
      sort(processes.begin(),processes.end(),[](const ProcessInfo &a, const ProcessInfo &b)
      {
         return a.cpuUsage > b.cpuUsage;
      });
   }
   else{
      sort(processes.begin(),processes.end(),[](const ProcessInfo &a, const ProcessInfo &b)
      {
         return a.memoryUsage > b.memoryUsage;
      });
   }
}

int main(){
   vector <ProcessInfo> processes = getAllProcesses();
   sortProcesses(processes,true);
   cout<<"PID\tCPU%\tMemory (kB)\tName\n";
   for (size_t i=0;i<min(processes.size(),size_t(10));i++){
      cout<<processes[i].pid<<"\t"<<processes[i].cpuUsage<<"%\t"<<processes[i].memoryUs
age<<"%\t"<<processes[i].name<<"%\n";
   }
   int targetPid;
   cout<<"Enter PID to kill: ";
   cin>>targetPid;
   if(targetPid > 0){
```

```
        if (kill(targetPid, SIGKILL) == 0){
            cout<<"Process "<< targetPid << " terminated successfully\n";
        }
        else{
            perror("Failed to kill the process");
        }
    }
    return 0;
}
```

Output:



Day 5: Implement real-time update feature to refresh data every few seconds.

Code:

```
#include <iostream>
#include <filesystem>
#include <fstream>
#include <sstream>
#include <algorithm>
#include <vector>
#include <dirent.h>
#include <unistd.h>
#include <csignal>
#include <thread>
#include <chrono>

using namespace std;

namespace fs=std::filesystem;

struct ProcessInfo{
   int pid;
   string name;
   double cpuUsage;
   long memoryUsage;
};

string readFileValue(const string &path){
   ifstream file(path);
   string value;
```

```
    if (file.is_open()){
        getline(file,value);
    }
    return value;
}

double getSystemUptime(){
    ifstream file("/proc/uptime");
    double uptime;
    if (file.is_open()){
        file >> uptime;
    }
    return uptime;
}

ProcessInfo getProcessInfo(int pid,double systemUptime){
    ProcessInfo pinfo;
    pinfo.pid = pid;
    ifstream file("/proc/" + to_string(pid) + "/stat");
    string line;
    long utime=0, stime=0, starttime=0;
    if(file.is_open()){
        getline(file,line);
        istringstream ss(line);
        string token;
        int count=0;

        while(ss >> token){
```

```
            count++;
            if(count==2) pinfo.name=token;
            else if(count==14) utime=stol(token);
            else if(count==15) stime=stol(token);
            else if(count==22) starttime=stol(token);
        }
    }

    ifstream memFile("/proc/" + to_string(pid) + "/status");
    if (memFile.is_open()){
        string key, value,unit;
        while(memFile >> key >> value >> unit){
            if (key=="VmRSS"){
                pinfo.memoryUsage=stol(value);
                break;
            }
        }
    }
    long total_time = utime + stime;
    double seconds = systemUptime - (starttime / sysconf(_SC_CLK_TCK));
    pinfo.cpuUsage = (total_time / sysconf(_SC_CLK_TCK)) / seconds * 100;
    return pinfo;


}

vector <ProcessInfo> getAllProcesses(){
    vector <ProcessInfo> processes;
```

```
    double systemUptime= getSystemUptime();
    for (const auto &entry : fs::directory_iterator("/proc")){
        if (entry.is_directory()){
            string filename=entry.path().filename().string();
            if (all_of(filename.begin(),filename.end(), ::isdigit)){
                int pid=stoi(filename);
                processes.push_back(getProcessInfo(pid,systemUptime));
            }
        }
    }
    return processes;
}

void sortProcesses(vector<ProcessInfo> &processes, bool sortByCpu){
    if(sortByCpu){
        sort(processes.begin(),processes.end(),[](const ProcessInfo &a, const ProcessInfo &b)
        {
            return a.cpuUsage > b.cpuUsage;
        });
    }
    else{
        sort(processes.begin(),processes.end(),[](const ProcessInfo &a, const ProcessInfo &b)
        {
            return a.memoryUsage > b.memoryUsage;
        });
    }
}
```

```
int main(){
    char input;
    while(true){
        system("clear");
        vector<ProcessInfo> processes = getAllProcesses();
        sortProcesses(processes, true); // Change to false to sort by memory usage

        cout << "PID\tCPU%\tMemory (kB)\tName\n";
        for (size_t i = 0; i < min(processes.size(), size_t(10)); ++i) {
            cout << processes[i].pid << "\t"
            << processes[i].cpuUsage << "\t"
            << processes[i].memoryUsage << "\t"
            << processes[i].name << "\n";
        }
        int targetPid;
        cout <<"Enter PID to Kill : " << endl;
        cin >> targetPid;
        //Signature : int kill(pid_t,int sig)
        if(targetPid){
            if(kill(targetPid,SIGKILL)==0){
                cout << "Process " << targetPid << " terminated successfully" << endl;
            }
            else{
                perror("Failed to kill Process");
            }
        }
        cout <<"\nPress 'q' to quit or Enter to refresh : ";
```

```
        cin.ignore();
        input = getchar();
        if(input == 'q' || input == 'Q')
            break;
        this_thread::sleep_for(chrono::seconds(2));
    }
    return 0;
}
```

Output: